作业 _Week15

20244227004 蔡博凯

2024 年 12 月 21 日

第一题　请简述稀疏线性编码器与稀疏自编码器训练方法的异同。

解答

相同点

- **稀疏性目标**：两者都通过引入稀疏性约束，使网络在表示数据时倾向于激活较少的神经元，从而提高特征的辨别能力。
- **优化目标**：损失函数均包含以下两部分：
  1. **重构误差**：用于最小化输入数据 $\mathbf{x}$ 与重构数据 $\hat{\mathbf{x}}$ 之间的差异：
  $$L_{\text{reconstruction}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2.$$
  2. **稀疏性约束**：通过稀疏正则化（如 L_1 正则化或 KL 散度）实现：
  $$L_{\text{sparsity}} = \lambda \sum_i \|\mathbf{z}_i\|_1 \quad \text{或} \quad \lambda \sum_i D_{\text{KL}}(p \| \hat{p}),$$
  其中 $\mathbf{z}_i$ 为编码后的激活值，D_{KL} 为 KL 散度。
- **无监督训练**：两者的训练过程均为无监督，输入与输出相同，目标是提取高效特征。

不同点

方面	稀疏线性编码器	稀疏自编码器
编码器结构	线性变换	包含非线性激活函数
稀疏性施加方式	直接施加在编码结果上	作用于隐藏层激活值
参数化	编码过程可以非参数化	网络权重为可学习参数
训练方式	优化问题求解（如凸优化）	通过反向传播训练
模型复杂性	简单，适用于线性任务	复杂，能捕捉非线性关系

表 1: 稀疏线性编码器与稀疏自编码器的主要区别

总结

稀疏线性编码器偏向简单线性特征提取，稀疏性直接施加在编码结果上；稀疏自编码器则结合了非线性特性，稀疏性主要体现在隐藏层激活上，能捕捉更复杂的特征关系。选择哪种方法主要取决于任务需求和数据复杂性。

第二题　请写出 VAE 训练的伪代码。

解答

Algorithm 1 变分自编码器（VAE）训练伪代码

Require: 训练数据 $\mathbf{X}$，编码器参数 θ，解码器参数 ϕ，训练轮数 N_{epochs}

初始化编码器参数 θ 和解码器参数 ϕ
for 轮次 $k = 1$ 到 N_{epochs} **do**
  for 每个小批量数据 **x do**
    前向传播：
      使用编码器计算潜在空间均值 μ 和对数方差 $\log \sigma^2$：$(\mu, \log \sigma^2) = \text{Encoder}(\mathbf{x}; \theta)$
      计算标准差 $\sigma = \exp(0.5 \cdot \log \sigma^2)$
      从标准正态分布中采样 $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$
      使用重参数化技巧计算潜在变量 $\mathbf{z} = \mu + \sigma \cdot \boldsymbol{\epsilon}$
      使用解码器生成重构数据 $\hat{\mathbf{x}} = \text{Decoder}(\mathbf{z}; \phi)$
    计算损失：
      重构损失：$\mathcal{L}_{\text{reconstruction}} = \text{Loss}(\mathbf{x}, \hat{\mathbf{x}})$
      KL 散度损失：$\mathcal{L}_{\text{KL}} = -0.5 \cdot \sum (1 + \log \sigma^2 - \mu^2 - \exp(\log \sigma^2))$
      总损失：$\mathcal{L} = \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{KL}}$
    反向传播：计算梯度 $\nabla_\theta \mathcal{L}$ 和 $\nabla_\phi \mathcal{L}$
    参数更新：使用优化器更新 θ 和 ϕ
    清空梯度：Optimizer.zero_grad()
  end for
  可选：记录当前轮次的总损失 $\mathcal{L}$
end for

第三题 请写出 GAN 训练的伪代码，并简述 GAN 训练困难的主要原因。

解答

GAN 训练伪代码

Algorithm 2 生成对抗网络（GAN）训练伪代码

Require: 真实数据分布 $p_{\text{data}}(x)$，生成器 $G(z;\theta_G)$，判别器 $D(x;\theta_D)$，学习率 α，训练轮数 N_{epochs}

初始化生成器参数 θ_G 和判别器参数 θ_D

for 轮次 $k = 1$ 到 N_{epochs} **do**

  for 每个小批量数据 $\{x^{(i)}\}$ **do**

    从真实数据分布 p_{data} 中采样真实样本 $\{x^{(i)}\}$

    从先验分布 $p_z(z)$ 中采样潜在变量 $\{z^{(i)}\}$

    生成样本：$\{\tilde{x}^{(i)}\} = G(z^{(i)};\theta_G)$

    训练判别器 D：

      最大化判别器的损失函数：

$$\mathcal{L}_D = -\frac{1}{m}\sum_{i=1}^{m}\left[\log D(x^{(i)}) + \log(1 - D(\tilde{x}^{(i)}))\right]$$

    更新判别器参数 $\theta_D \leftarrow \theta_D - \alpha\nabla_{\theta_D}\mathcal{L}_D$

    训练生成器 G：

      最大化生成器的损失函数（等价于最小化 $-\log D(G(z))$）：

$$\mathcal{L}_G = -\frac{1}{m}\sum_{i=1}^{m}\log D(G(z^{(i)}))$$

    更新生成器参数 $\theta_G \leftarrow \theta_G - \alpha\nabla_{\theta_G}\mathcal{L}_G$

  end for

end for

GAN 训练的困难原因

- **模式崩溃（Mode Collapse）**：生成器可能只学习到部分真实数据分布，生成有限种类的样本。
- **不稳定性（Instability）**：判别器和生成器的优化目标相互对抗，可能导致训练过程振荡或难以收敛。
- **梯度消失（Vanishing Gradient）**：当判别器性能过强时，生成器的梯度可能接近零，从而无法有效更新。
- **优化难度（Optimization Difficulty）**：GAN 训练是一个双玩家极小极大博弈，优化复杂且不易控制。
- **评估困难（Evaluation Difficulty）**：缺乏统一的指标来衡量生成样本质量和训练效果。

第四题　请简述在 Score matching 模型中如何解决数据密度低的问题。

解答

在 Score Matching 模型中，当数据密度较低时，直接估计分布的梯度可能会变得不准确，从而导致模型性能下降。以下是几种常见的解决方法：

1. 重加权密度估计

为了提高对低密度区域的估计质量，可以对损失函数进行加权。假设数据分布为 $p(x)$，损失函数为：

$$\mathcal{L} = \mathbb{E}_{p(x)} \left[\frac{1}{2} \|\nabla_x \log p(x) - \nabla_x \log q(x)\|^2 \right],$$

通过引入权重函数 $w(x)$，对低密度区域赋予更高的权重：

$$\mathcal{L}_w = \mathbb{E}_{p(x)} \left[w(x) \cdot \frac{1}{2} \|\nabla_x \log p(x) - \nabla_x \log q(x)\|^2 \right],$$

其中 $w(x)$ 通常与密度的倒数成正比。

2. 使用加噪数据

通过在数据上添加随机噪声，使得分布更加平滑，减少低密度区域的影响。例如，定义加噪数据分布为：

$$p_{\text{noisy}}(x) = \int p(x-\epsilon) \cdot \mathcal{N}(\epsilon; 0, \sigma^2 I)\, d\epsilon,$$

然后对加噪数据分布进行梯度估计。

3. 正则化

在损失函数中加入正则化项，限制模型在低密度区域的行为，例如添加 L_2 正则化：

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \|\theta\|^2,$$

其中 λ 是正则化强度的超参数，θ 是模型参数。

4. 双重估计

结合隐式和显式密度估计方法，将模型梯度分为两部分处理，增强模型对低密度区域的鲁棒性。